

Digital System Design

LAB V

Seven-Segment Display

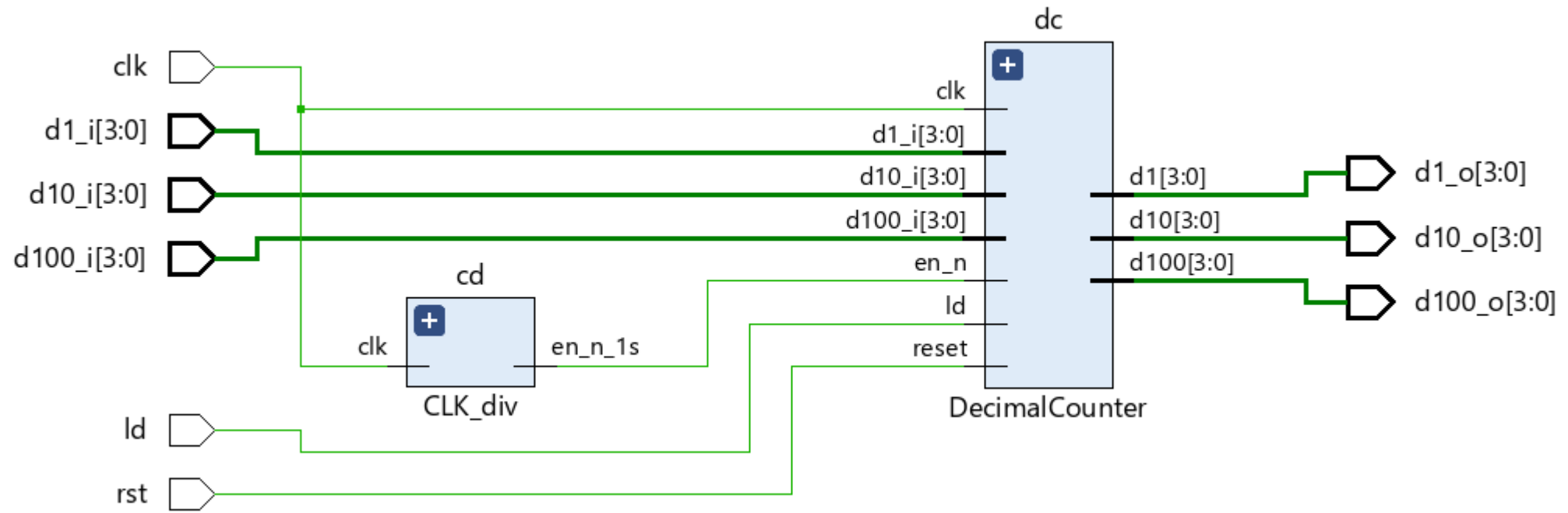
DONG Yunyang

dongyy@sustech.edu.cn

411, No. 2, Hui Yuan

Tencent Meeting: 874-068-9694

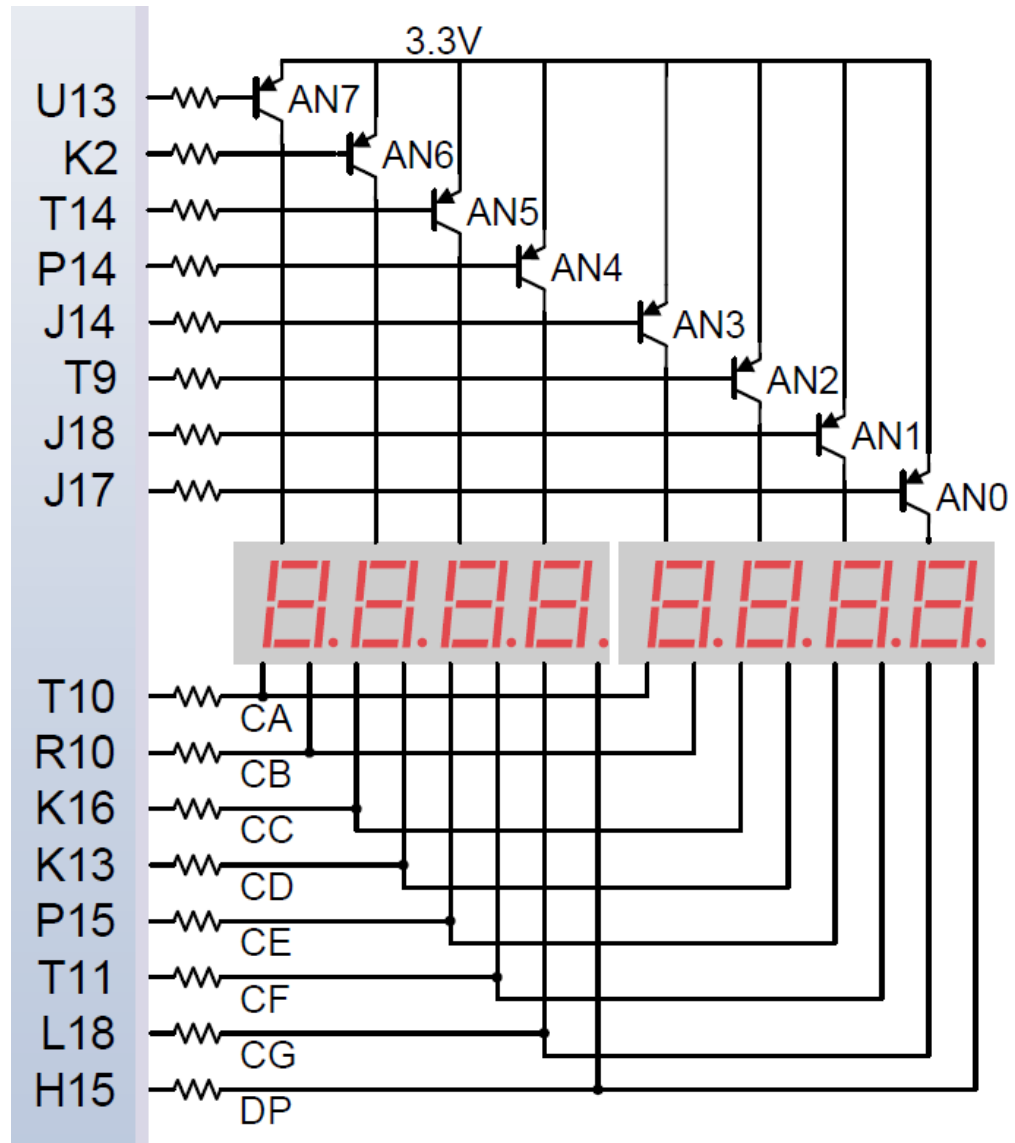
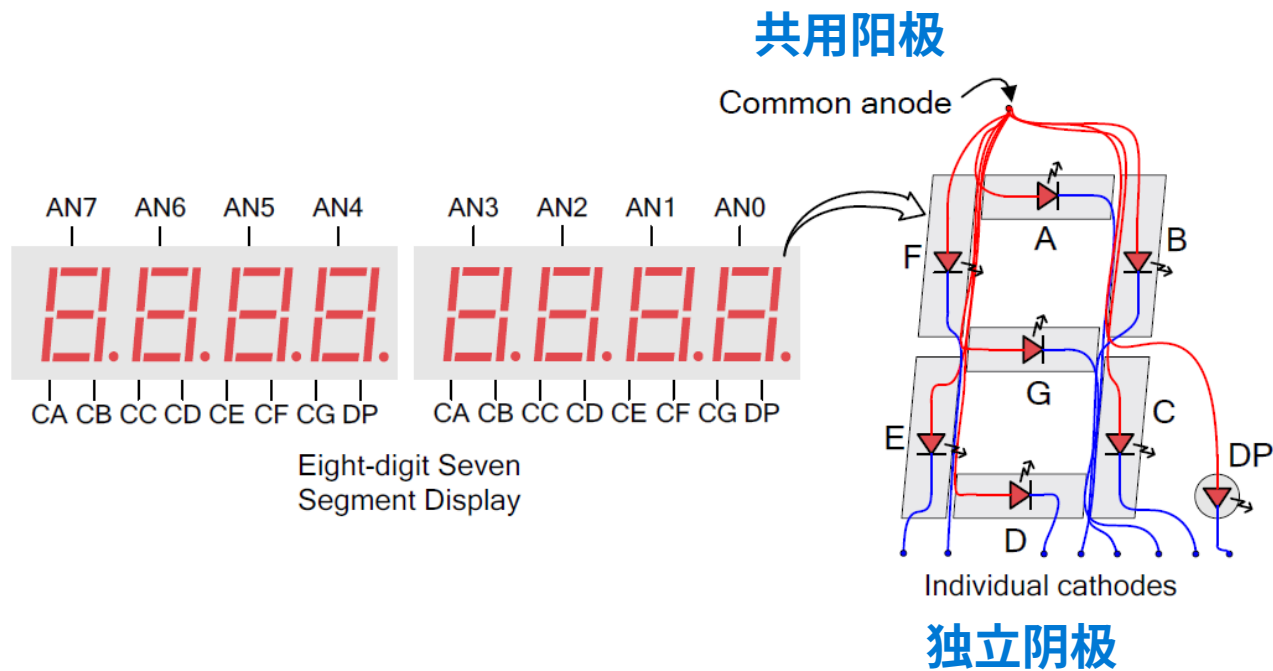
Review – Decimal Counter



Seven-Segment Display

七段数码管是数字系统中字母、数字常用显示工具。

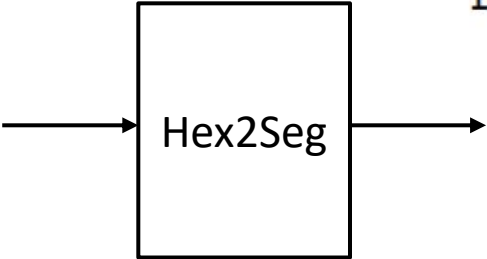
Nexys DDR 4 开发板上有八位七段数码管。每个七段数码管由七个条形发光二极管和一个圆形发光二极管（用作小数点）组成，数码管电路连接方式如图。



Seven-Segment Decoder

通过点亮发光二极管的不同组合，它可以显示不同的数字和字母。

通过设计一个十六进制到七段数码管的解码器，接收一个 4 位的输入（一个十六进制数字），输出相应的 8 位编码（包括小数点），从而点亮对应的发光二极管段。



x	a	b	c	d	e	f	g
0	0	0	0	0	0	0	1
1	1	0	0	1	1	1	1
2	0	0	1	0	0	1	0
3	0	0	0	0	1	1	0
4	1	0	0	1	1	0	0
5	0	1	0	0	1	0	0
6	0	1	0	0	0	0	0
7	0	0	0	1	1	1	1
8	0	0	0	0	0	0	0
9	0	0	0	0	1	0	0
A	0	0	0	1	0	0	0
B	1	1	0	0	0	0	0
C	0	1	1	0	0	0	1
D	1	0	0	0	0	1	0
E	0	1	1	0	0	0	0
F	0	1	1	1	0	0	0

1 = off

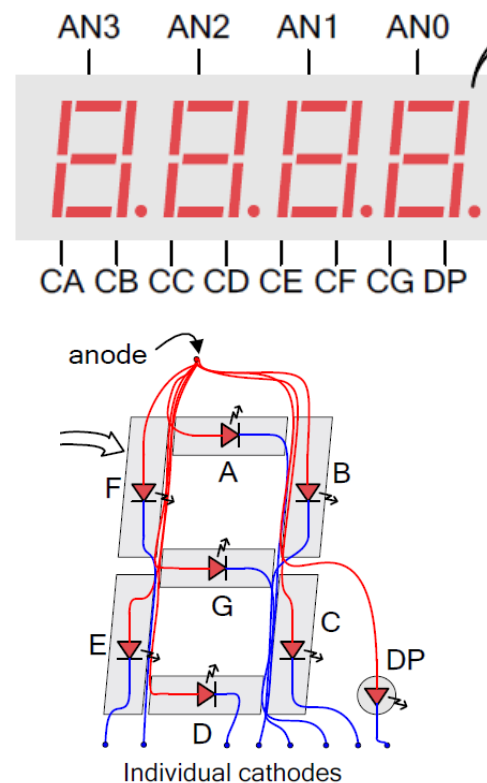
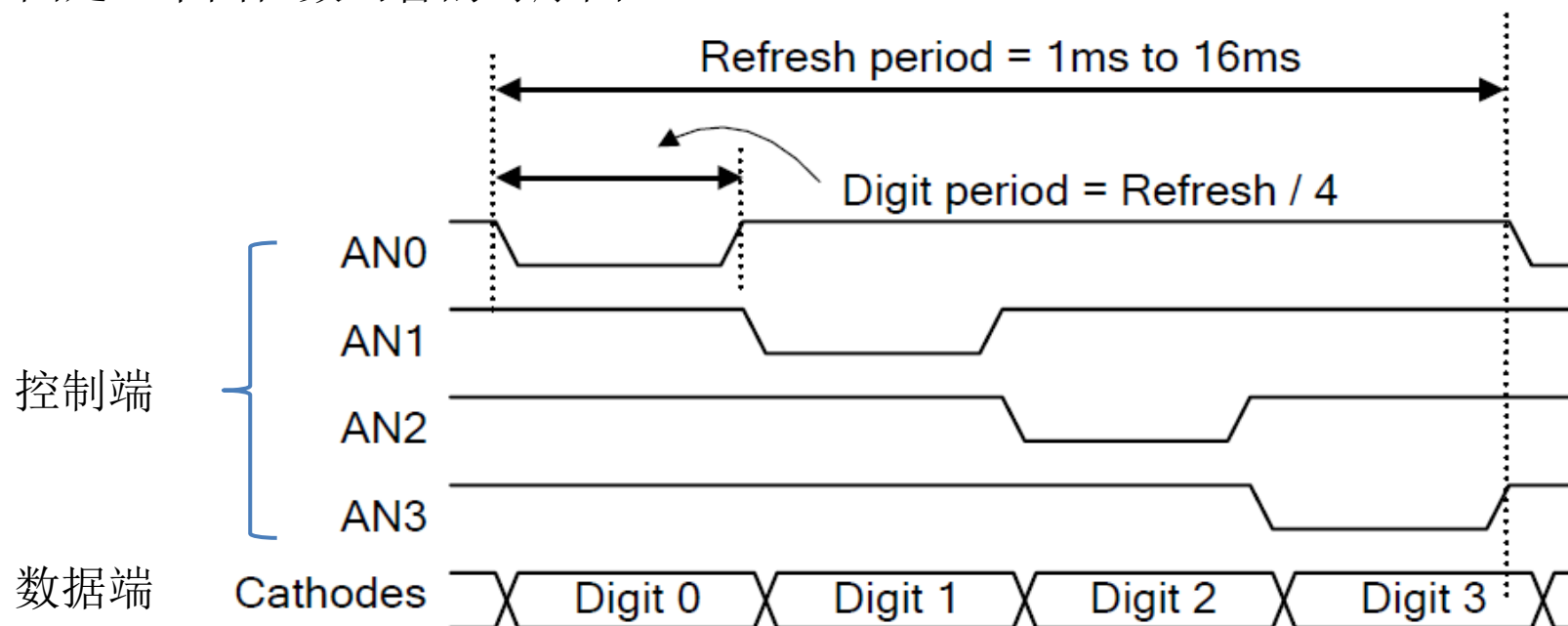
0 = on

Multiplexing Display

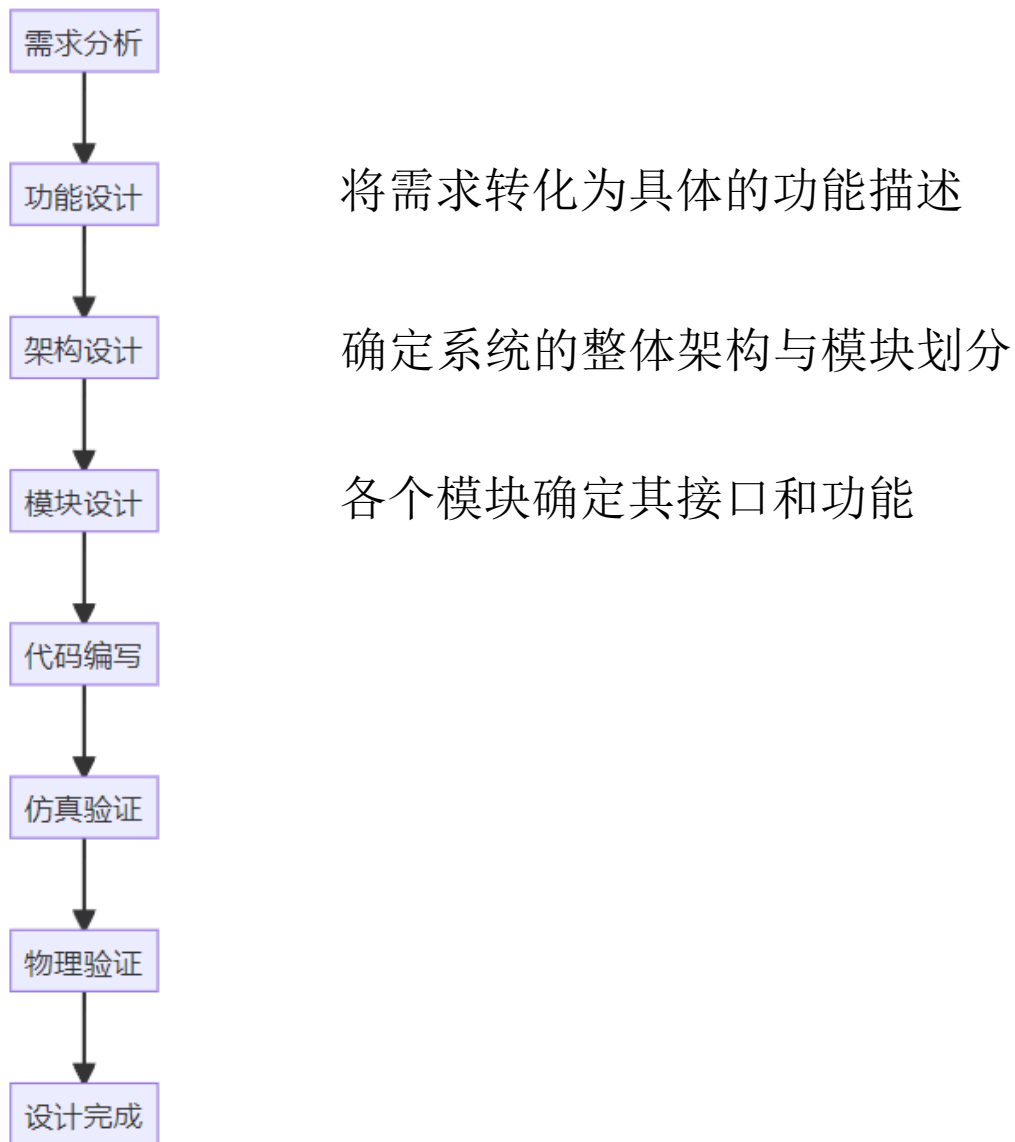
通过将单个数码管共用阳极端，八位数码管共用数据端的方式，大大节省了FPGA需要的引脚。

我们可以通过足够快的刷新速率对八位数码管的阳极控制线进行时分复用，来“同时”显示八个不同的字符。由于视觉暂留效应，人眼会认为所有的八个数码管都处于点亮状态，并且显示出正确的值。如果刷新率降低到大约 45 Hz，人们就会觉得数字开始闪烁。

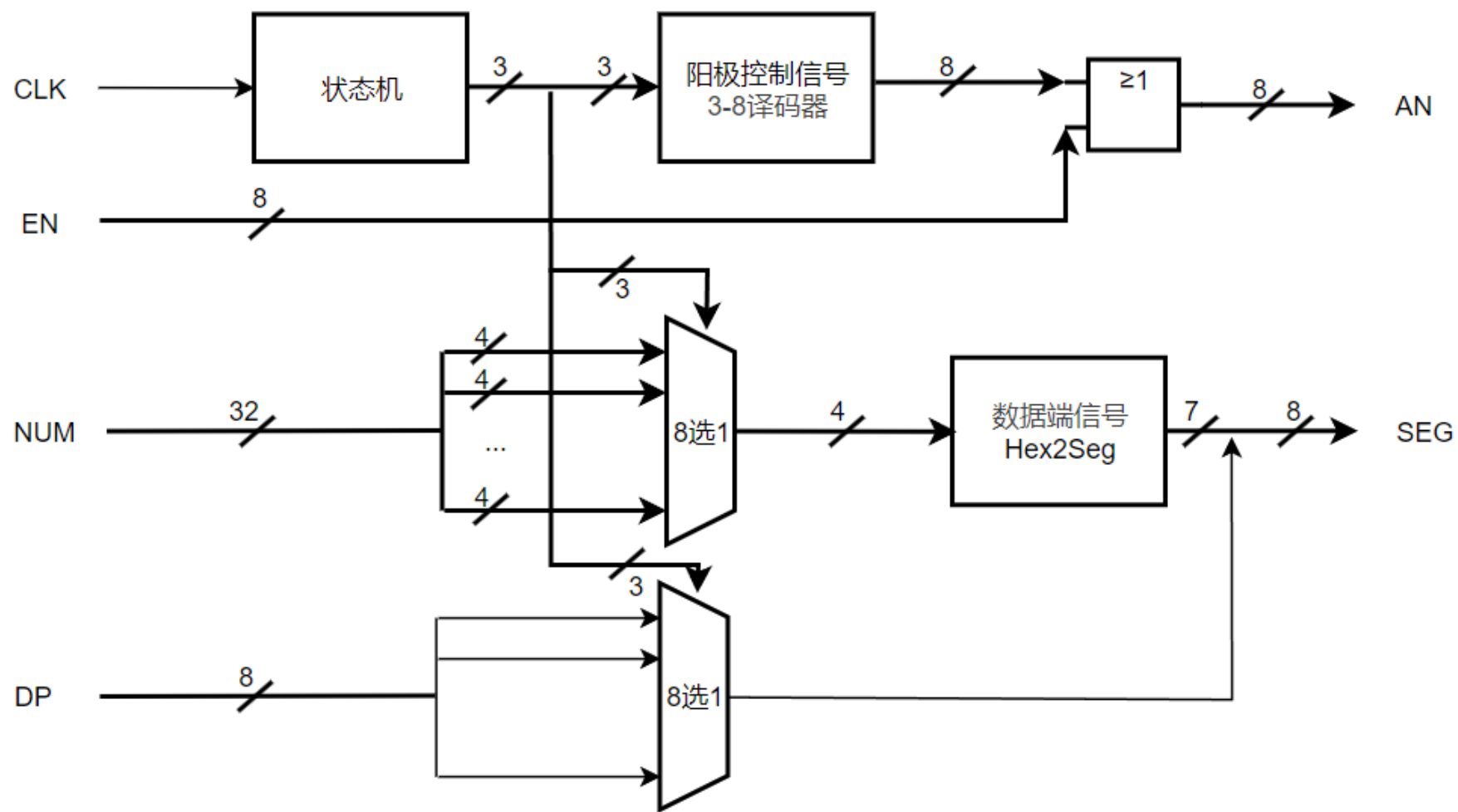
下图是一个四位数码管的时序图。



Digital System Design



Architecture



VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
use IEEE.std_logic_arith.all;

entity SevenSegDisplay is
    Port(clk : in  std_logic;                -- 100MHz系统时钟
          en_n : in std_logic_vector (7 downto 0); -- 显示使能端，低电平使能
          dp : in std_logic_vector (7 downto 0); -- 小数点控制端
          number : in  std_logic_vector (31 downto 0); -- 8个数字的输入数据(每个数字4位，共8*4=32位)
          seg : out  std_logic_vector (7 downto 0); -- 数据端(最高位为小数点)
          an : out  std_logic_vector (7 downto 0)); -- 阳极选择端
end SevenSegDisplay;

architecture Behavioral of SevenSegDisplay is

    signal cnt: std_logic_vector (19 downto 0);
    signal anode_select: std_logic_vector (2 downto 0); -- 3位阳极扫描信号，也是状态机状态
    signal digit_data: std_logic_vector (3 downto 0); -- 当前显示的数字数据
    signal dp_bit: std_logic; -- 当前小数点状态
    signal segment_data: std_logic_vector (6 downto 0); -- 七段数码管数据信号(不含小数点)
    signal an_tmp: std_logic_vector (7 downto 0); -- 未与使能端组合前的阳极扫描信号(8位)
```


VHDL

```
begin
    -- 计数器
    process(clk)
    begin
        if rising_edge(clk) then
            cnt <= cnt + '1';
        end if;
    end process;

    -- 最高三位，作为扫描状态（每个数字刷新率：100MHz/2^20 ≈ 95.3Hz）
    anode_select <= cnt(19 downto 17);
```

```
-- 阳极扫描（3-8译码器）
with anode_select select
    an_tmp <=
        "11111110" when "000",
        "11111101" when "001",
        "11111011" when "010",
        "11110111" when "011",
        "11101111" when "100",
        "11011111" when "101",
        "10111111" when "110",
        "01111111" when "111",
        "11111111" when others;

-- 应用低电平使能控制
an <= an_tmp or en_n;
```

VHDL

-- 数据选择多路复用

with anode_select select

digit_data <=

```
number(3 downto 0) when "000",
number(7 downto 4) when "001",
number(11 downto 8) when "010",
number(15 downto 12) when "011",
number(19 downto 16) when "100",
number(23 downto 20) when "101",
number(27 downto 24) when "110",
number(31 downto 28) when "111",
"0000" when others;
```

-- 小数点位

with anode_select select

dp_bit <=

```
dp(0) when "000",
dp(1) when "001",
dp(2) when "010",
dp(3) when "011",
dp(4) when "100",
dp(5) when "101",
dp(6) when "110",
dp(7) when "111",
'1' when others;
```

-- 十六进制到七段数码管译码 (g fed cba)

with digit_data select

segment_data <=

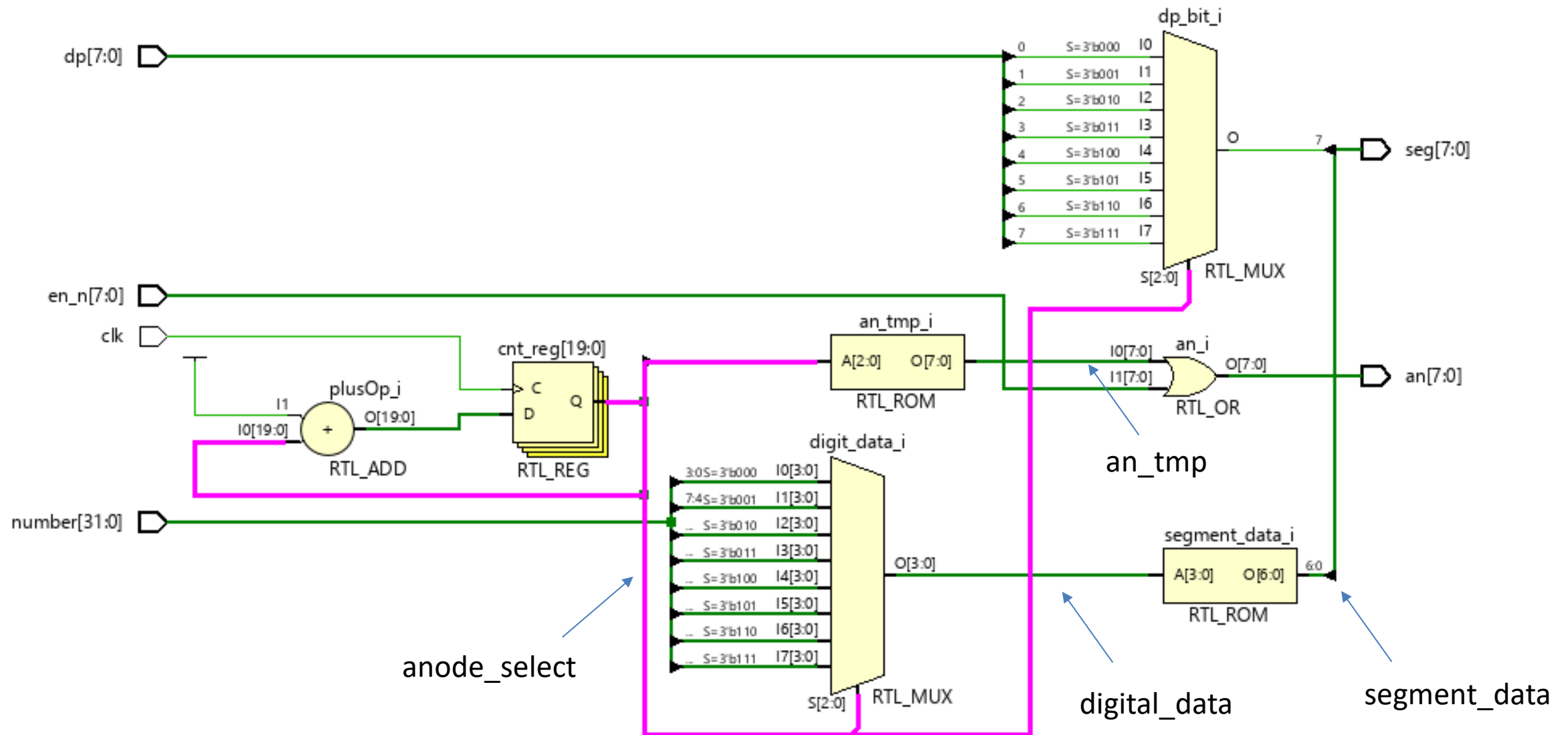
```
"1000000" when "0000", -- 0
"1111001" when "0001", -- 1
"0100100" when "0010", -- 2
"0110000" when "0011", -- 3
"0011001" when "0100", -- 4
"0010010" when "0101", -- 5
"0000010" when "0110", -- 6
"1111000" when "0111", -- 7
"0000000" when "1000", -- 8
"0010000" when "1001", -- 9
"0001000" when "1010", -- A
"0000011" when "1011", -- b
"1000110" when "1100", -- C
"0100001" when "1101", -- d
"0000110" when "1110", -- E
"0001110" when "1111", -- F
"1111111" when others; -- 全灭
```

-- 组合小数点和七段数据 (小数点在最高位)

seg <= dp_bit & segment_data;

end Behavioral;

RTL Schematic



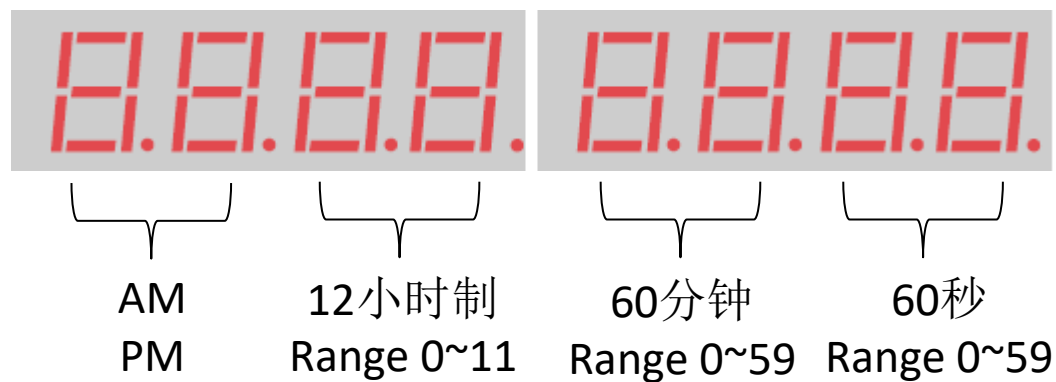
Exercise

请仔细阅读数据手册中关于数码管的内容，充分理解数码管显示原理和程序。
编写一个顶层程序，数码管上从左到右显示为56789.abc

Assignment 1

请设计一个数字钟，并用数码管显示时间。

8个数码管按下图分配



例如



(可以不用小数点分隔)

Assignment 1

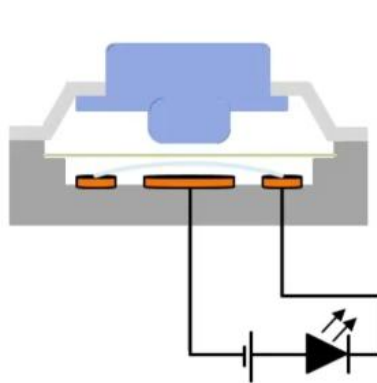
数字钟需具有以下功能：

1. 每1s计数1次。
2. 用8个数码管显示结果。
3. 使用BTNC作为设置位选按键。按下后循环切换“0.空闲” - “1.设置分” - “2.设置小时” - “3.设置上下午” - “0.空闲”状态。其中1~3状态时计数暂停，对应位置数字进行闪烁（1s亮暗各1次）；0状态为正常计数状态。
4. 使用BTNU作为递增按键。在1~3状态中，按下按键后数字+1，0状态下无功能。
5. BTNC和BTNU按键需要消抖和避免一次按键多次触发，设计见后续。
6. 推荐的模块设计见后续。

本题需要验收，请演示以上所有功能。

Assignment 1

- 按键消抖



```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;

entity ButtonDebounce is
    Port (
        clk : in std_logic;           -- 系统时钟(100MHz)
        btn_in : in std_logic;        -- 按键输入
        btn_out : out std_logic       -- 消抖后的按键输出, 按键抬起后输出1次
    );
end ButtonDebounce;

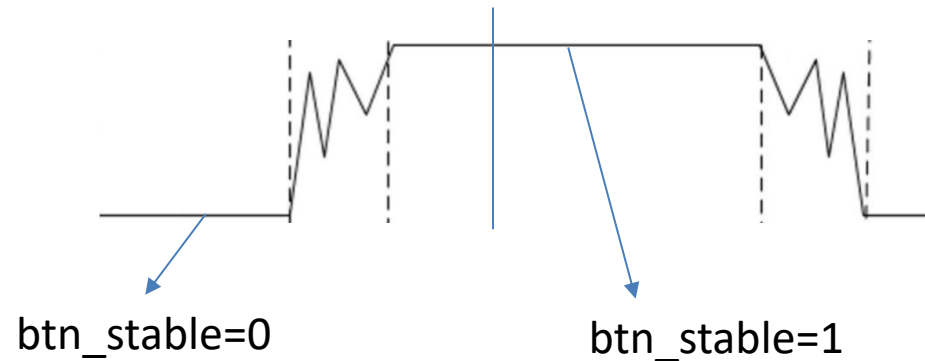
architecture Behavioral of ButtonDebounce is
    signal btn_prev : std_logic := '0'; -- 存储上一个稳定状态, 用于边沿检测
    signal btn_sync : std_logic_vector(1 downto 0) := "00"; -- 将按键信号同步到时钟信号
    signal counter : std_logic_vector(20 downto 0) := (others => '0'); -- 消抖计数器
    signal btn_stable : std_logic := '0'; -- 消抖后的稳定信号
```

Assignment 1

- 按键消抖

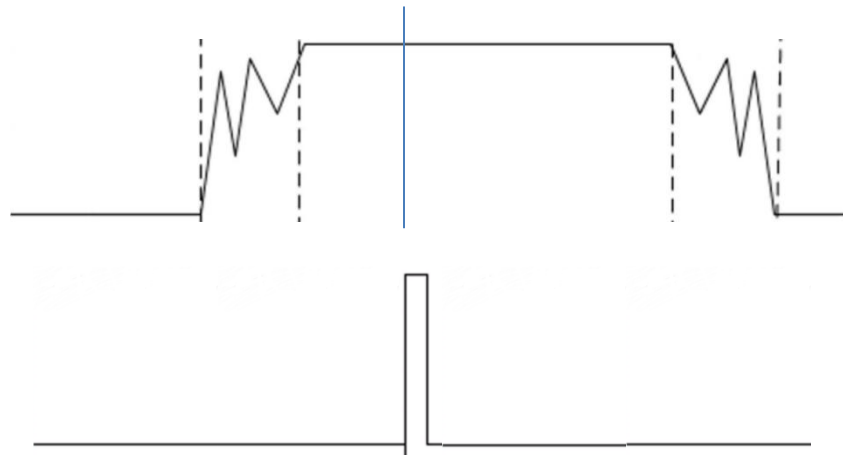
```
-- 将按键信号同步到时钟
process(clk)
begin
    if rising_edge(clk) then
        btn_sync <= btn_sync(0) & btn_in; -- 移位寄存器, btn_in进入低位, 低位移动至高位, 用以消除亚稳态
    end if;
end process;
```

```
-- 消抖计数器
process(clk)
begin
    if rising_edge(clk) then
        if btn_sync(1) /= btn_stable then -- 当输入与当前稳定状态不同时
            if counter = 2_000_000 then -- 计数到2,000,000 (20ms)
                btn_stable <= btn_sync(1); -- 更新稳定状态为当前输入
                counter <= (others => '0');
            else
                counter <= counter + 1;
            end if;
        else
            counter <= (others => '0'); -- 输入按键与稳定状态相同, 重置计数器
        end if;
    end if;
end process;
```



Assignment 1

- 按键消抖



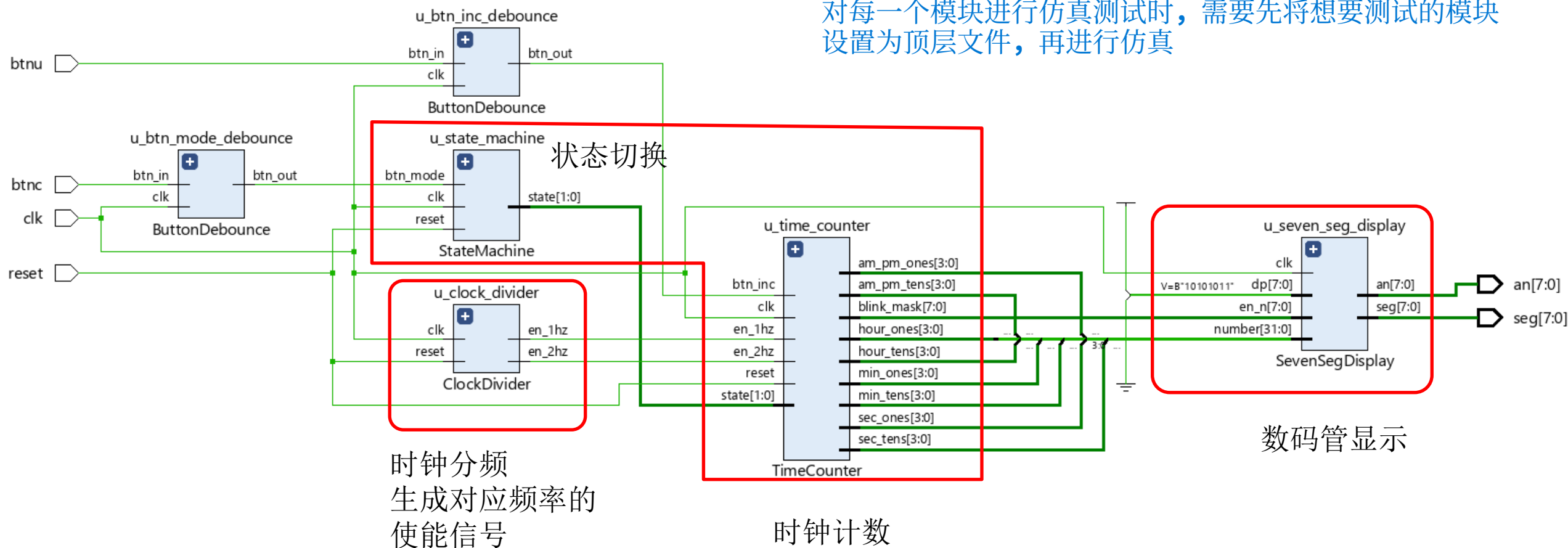
```
-- 边沿检测
process(clk)
begin
    if rising_edge(clk) then
        btn_prev <= btn_stable; -- 保存当前稳定状态用于下次比较
        if btn_prev = '0' and btn_stable = '1' then -- 前一次是0, 当前是1, 检测到按键按下的上升沿
            btn_out <= '1'; -- 上升沿时输出一个时钟周期的1, 其他时候都输出0, 保证一次按键只触发一次
        else
            btn_out <= '0';
        end if;
    end if;
end process;
end Behavioral;
```

Assignment 1

写成多个文件的形式
在顶层将模块连接在一起

- 程序框图

对每一个模块进行仿真测试时，需要先将想要测试的模块
设置为顶层文件，再进行仿真



Assignment 2

- 斐波那契数列

斐波那契数列定义如下：

$$fib(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ fib(n-1) + fib(n-2) & n > 1 \end{cases}$$

请用VHDL实现斐波那契数列计算。其中 n 为6位二进制输入，视为无符号整数。 $fib(63)=6557470319842$

Assignment 2

1. 请推导算法状态机图(ASM chart);
2. 请编写 VHDL 代码;
3. 请编写testbench文件对设计进行仿真;
4. 请展示post implementation simulation results。

提交简易报告，包括：题目、算法状态机图、VHDL关键代码和testbench关键代码（合理注释）、结果（RTL Schematic，仿真波形）